RSCL — Algorithm 7: Proximity Watchdog, Hold Rules and Parked-Aircraft Handling

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

hu-telepresence-gazebo/festival/control_server_hu.py	sha256 662fd7191f00	2026-11-12 02:11 UTC	clean (HEAD b12b0f0; last change e14ef3f)
hu-telepresence-gazebo/acceptance/strategist.py	sha256 33751f6df444	2026-11-12 02:07 UTC	clean (last change f50621f)

1. Purpose and context

All planning in the demo (Algorithms 5, 6, 8) is done on idealised straight-line paths. The proximity watchdog is the runtime backstop in the supervisor's 10 Hz control loop: it observes actual odometry and stops a manoeuvring aircraft that gets too close to another aircraft. Around it are a set of mode rules that keep parked aircraft disarmed, prevent "hidden" take-offs from hold or fault-clear commands, and gate single-aircraft relative moves with a static path check.

2. Inputs, outputs, state

Per tick inputs: for each aircraft, odometry position/velocity/yaw with receive time, current mode, setpoint, fault flag. **Outputs:** velocity commands on `/<name>/gazebo/command/twist`, motor enable/disable on `/<name>/enable`, mode changes and events (`SEPARATION_HOLD`, `LANDED`). **Modes:** parked = {READY, GROUNDED, WAITING, QUEUED}; manoeuvring = {TAKEOFF, MOVE, FORMATION, TURN, SPIN}; others HOVER, LANDING, FAULT.

3. Algorithm

Algorithm 7.1 Watchdog and parked handling (per tick, before control)

Source: festival/control_server_hu.py: Fleet.loop

```
every 0.10 s, under the fleet lock, for each aircraft d do
  if not fresh(d) then send zero velocity without enable; continue            // telemetry watchdog (age ≥ 2.0 s)
  if d.mode = QUEUED then apply release rule (Algorithm 6.2); continue
  if d.mode ∈ {READY, GROUNDED, WAITING} then send enable = false; continue  // actively disarmed at 10 Hz
  if d.mode ∈ {FORMATION, MOVE, TAKEOFF} then                                 // LANDING, HOVER, TURN, SPIN excluded
    C ← { o ≠ d : fresh(o) and not (d.mode = TAKEOFF and o.mode ∈ parked) and ‖p_d − p_o‖ < 0.82 }
    if C ≠ ∅ then
      d.target ← p_d; send zero velocity (enabled)
      if d.z < 0.30 then send enable = false; d.mode ← GROUNDED; d.target_z ← 0.10   // never a false HOVER on the ground
      else d.mode ← HOVER
      emit SEPARATION_HOLD(d, C); continue
  if d.mode = FAULT then send zero velocity; continue
  if d.mode = LANDING and d.z ≤ 0.20 and |d.vz| < 0.30 then
    send zero velocity; send enable = false; d.mode ← GROUNDED; emit LANDED; continue
  ... setpoint tracking and arrival detection (Algorithm 9)
```

Algorithm 7.2 Command-time hold rules and single-aircraft move admission

Source: festival/control_server_hu.py: Fleet.command

```
procedure ADMIT(action, targets, params)
  if action ∉ {clear_fault, land} and not emergency_hover and any target faulted then refuse
  stale ← targets without fresh odometry
  if stale ≠ ∅ then
    if action ∉ {land, hover} then refuse                    // risk-increasing actions fail closed
    targets ← targets \ stale (refuse if empty); log SAFETY_SKIPPED
  if action ∈ {move, turn, spin} and any target ∉ {TAKEOFF, HOVER, MOVE, FORMATION, TURN} then refuse
  if action = move and |targets| = 1 then
    require d.mode = HOVER
    require no other aircraft in {MOVE, FORMATION, TAKEOFF, LANDING, SPIN}
    require all of dx, dy, dz, forward, right finite and |·| ≤ 2.0 m
    goal ← p_d + (dx + cosψ·fwd + sinψ·right,  dy + sinψ·fwd − cosψ·right,  dz)   // ENU, body-right =
(sinψ, −cosψ)
    require |goal.x|, |goal.y| ≤ 3.35 and 0.65 ≤ goal.z ≤ 2.8
    for each other aircraft o do
      require fresh(o)
      for k = 0..50: if ‖p_d + (goal − p_d)·k/50 − p_o‖ < 1.0 then refuse
  hover on parked or FAULT: target_z ← z, mode unchanged (no take-off, no fault bypass)
  clear_fault on ground (parked or z < 0.3): hold position; FAULT → GROUNDED; else HOVER at max(0.6, z)
```

4. Flowchart and state diagram

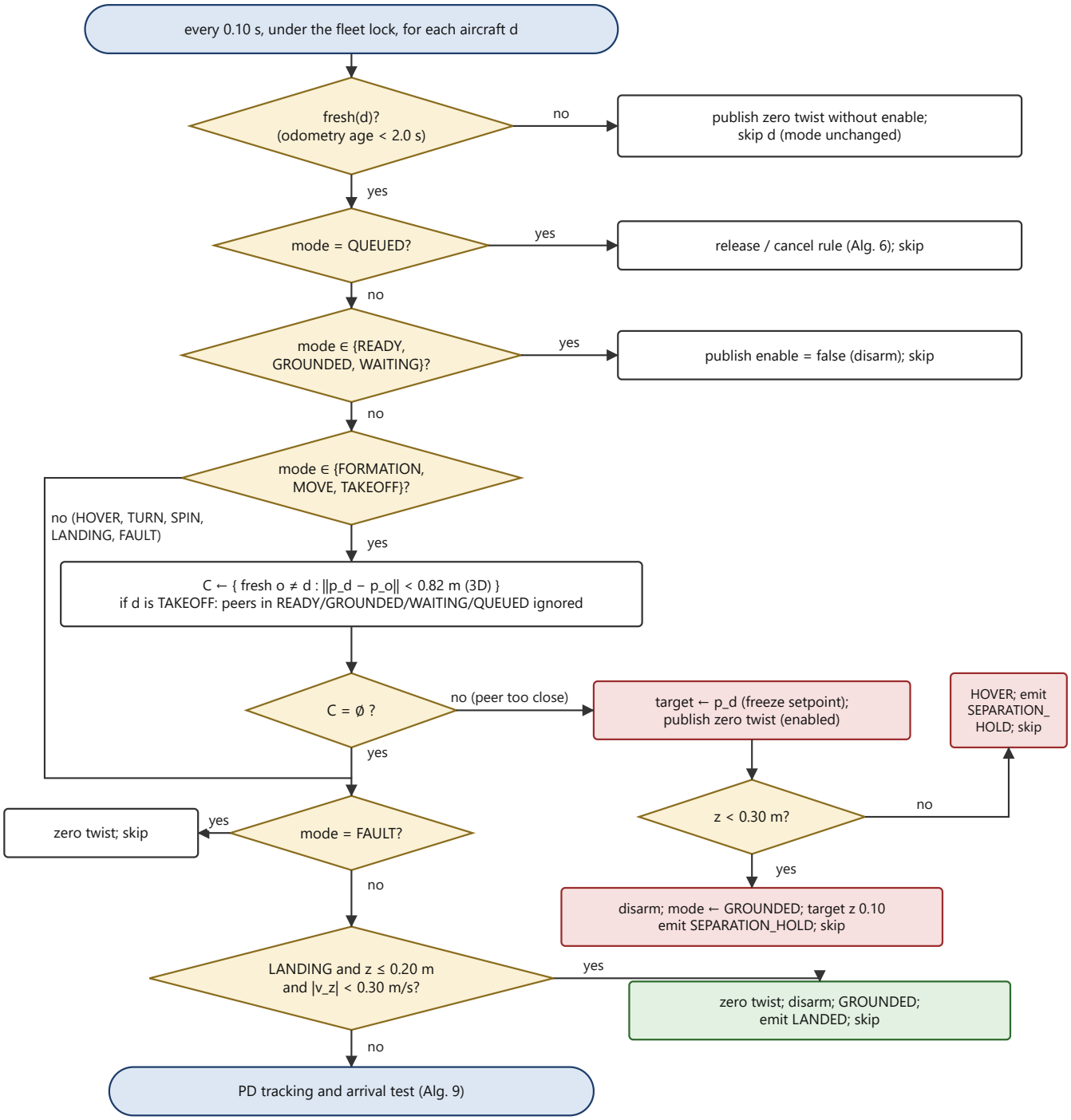


Figure 1. Per-tick flowchart of the telemetry watchdog, parked handling and proximity hold in `Fleet.loop`, executed for every aircraft every 0.10 s before setpoint tracking. Red boxes are the proximity-hold branch. LANDING is deliberately not subject to the hold.

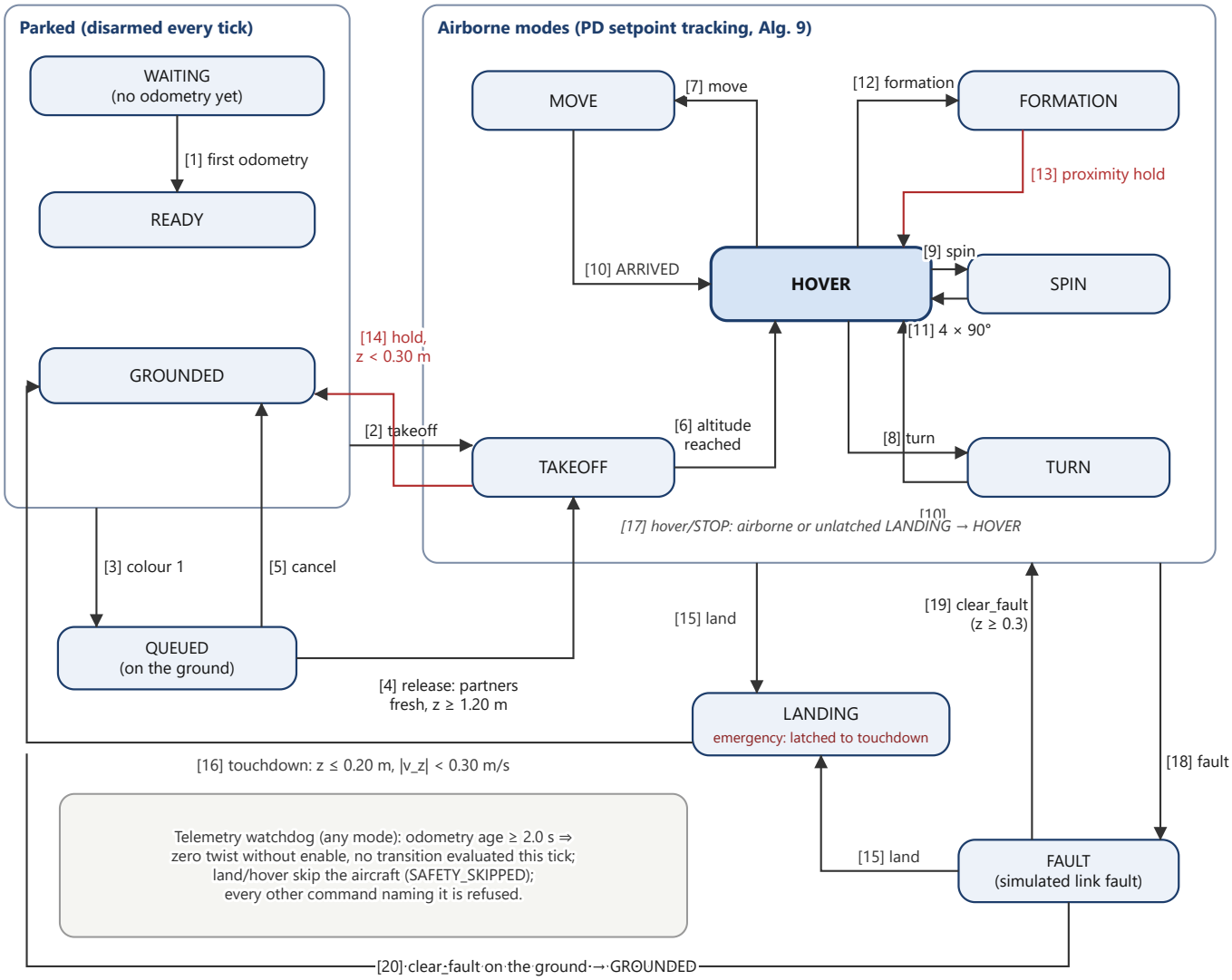


Figure 2. Supervisor per-drone mode FSM (`control_server_hu.py`). The proximity-hold transitions [13] and [14] are drawn in red. Bracketed numbers refer to the guard table in the Algorithm 9 document; the guards relevant here are: [13] fresh peer within 0.82 m while in FORMATION, MOVE or TAKEOFF (TAKEOFF ignores parked and QUEUED peers), $z \geq 0.30$ m → HOVER; [14] the same with $z < 0.30$ m → GROUNDED and disarmed; [15] land from any airborne mode or FAULT; [16] touchdown $z \leq 0.20$ m and $|v_z| < 0.30$ m/s → GROUNDED; [17] is not taken by an aircraft under the emergency-landing latch (set by `emergency_land`; while LANDING, every command except land, fault and clear_fault is ignored for it and logged `EMERGENCY LANDING LATCHED`; Algorithm 2). Telemetry staleness is not a mode: it suspends all transitions.

5. Parameters

Parameter	Value
Proximity hold distance	0.82 m, 3D Euclidean, strict
Modes subject to hold	FORMATION, MOVE, TAKEOFF (TAKEOFF ignores parked peers)
Ground threshold for hold	$z < 0.30$ m → GROUNDED, disarmed, target z 0.10
Touchdown	$z \leq 0.20$ m and $\lvert v_z \rvert < 0.30$ m/s
Telemetry freshness	last odometry < 2.0 s ago
Single-move admission	|component| ≤ 2.0 m; arena |x|,|y| ≤ 3.35 m; $0.65 \leq z \leq 2.8$ m; 51 path samples; 1.0 m to every other aircraft

Loop period	0.10 s (plus lock contention)
Observer thresholds (SpaceOps, display only)	alert < 0.85 m, watch < 1.15 m

6. Complexity

Watchdog: $O(N^2)$ distance checks per tick (N = 5: 20). Move admission: O(51·N).

7. Properties

Guaranteed

- Whenever a fresh manoeuvring aircraft (FORMATION/MOVE/TAKEOFF) is observed within 0.82 m of a fresh peer at a tick, its setpoint is frozen at its current position within that tick and it leaves the manoeuvring mode (it will not resume the old goal by itself).
- Landings are never interrupted by a proximity hold; emergency landings of close aircraft proceed. An emergency landing is in addition latched against later commands (Algorithm 2): neither the watchdog nor a later stop/hover, take-off or move can cancel it before touchdown.
- An aircraft below 0.30 m is never reported as HOVER by a hold; it is disarmed and reported GROUNDED.
- Parked aircraft receive an explicit disarm every tick, so a just-spawned model held by the velocity controller at its spawn height is not left hovering while reported as parked.
- Hold, land and fault-clear never launch a parked aircraft; a faulted aircraft accepts only land, clear_fault, or the emergency hover.
- A single relative move is admitted only from HOVER, with the rest of the fleet not manoeuvring, and with every sampled straight-line point ≥ 1.0 m from every other aircraft's current position.

Not guaranteed

- **Not a collision-avoidance guarantee.** Freezing the setpoint does not stop the aircraft instantly; with velocities up to 0.62 m/s per horizontal axis, distance can still shrink after detection. Both aircraft are not necessarily held: only aircraft in the listed modes are; a HOVER, TURN or SPIN aircraft is never held, and a HOVER peer does not move away.
- Detection is sampled at 10 Hz on odometry; a close pass between ticks is not seen.
- Aircraft with stale telemetry are invisible to the watchdog (excluded from C) and receive zero velocity with motors not re-enabled; their real position is unknown.
- A held aircraft is not re-planned; the operator or Algorithm 11 must decide the next step.
- Path admission uses other aircraft's current positions only (assumed stationary); concurrent manoeuvres are prevented by the mode rule, not by prediction.
- **Measured:** after holds, the observed separation continued to shrink to 0.633 m and 0.551 m (Section 8).

8. Testing and measured results

- Offline: `acceptance/tests/test_v9.py` (`test_move_collision_refused_without_mutation`, `test_move_valid_short_single`, `test_simulated_ground_stop_does_not_takeoff`, `test_existing_fault_refusal_not_bypassed`, `test_no_out_of_bounds_clamping`); `test_missions.py::test_fault_blocks_formation`.
- Live: in the 25-shape catalogue run (`formation_catalog_20261111T053652Z.json`, `...053814Z.json`) the watchdog recorded 0 proximity holds; fleet test minimum observed separation 0.998 m; swarm test 1.105 m; `motion_test.py` and `flight_test.py` PASS (run `run_20261111T052914Z`).
- **Live formation validation, 2026-11-12** (`acceptance/results/catalog_new/formation_catalog_20261112T011849Z.json` and

`catalog_new2/formation_catalog_20261112T012039Z.json`; VALIDATED requires settling < 30 s, RMS slot error ≤ 0.10 m, no proximity hold and observed separation ≥ 0.82 m). In the first run 44 transitions were VALIDATED with 0 holds. The transition `pentomino_f` → `pentomino_w` had a planned straight-line minimum separation of 0.884 m (accepted, since MIN_SEP is 0.88 m) and produced 3 proximity holds (event log 01:17:43 Ra/Osiris, 01:17:45 Anubis/Ra); observed minimum separation 0.633 m (Ra–Anubis), shape error 0.55 m, status EXPERIMENTAL. In the second run the first transition, from that held and distorted state to `line` (planned 0.894 m), again produced 3 holds (01:19:23 Ra/Anubis, 01:19:26 Thoth/Anubis), observed 0.551 m, EXPERIMENTAL; the remaining entries of that run are recorded as REFUSED (safe). Both observed minima are below the 0.82 m hold distance: the hold stopped new setpoint motion, but the distance kept shrinking after detection. Following the first hold, the strategist veto margin (`acceptance/strategist.py`, `VETO_M`, Algorithm 14) was raised to 1.10 m of planned straight-line clearance, with one exemption: a candidate is not vetoed if its tightest planned pass is no tighter than the starting spacing (veto only when the pass is < 1.10 m and more than 0.02 m below the starting separation); the supervisor's 0.82 m hold distance and MIN_SEP = 0.88 m are unchanged.

- The supervisor event log (`logs/hu_festival_events.jsonl`, 2840 lines on 2026-11-12) contains 14 `SEPARATION_HOLD` events (e.g. Horus/Ra/Thoth at 15:34:23 and the six above); the log does not record the distance at detection, so stopping distance after a hold is measured only through the catalogue's observed minima above.

9. Comparison with alternatives and related work

Runtime inter-robot collision avoidance is usually done by changing each robot's velocity command continuously, either geometrically (velocity obstacles and their reciprocal variant ORCA) or by a safety filter over a dynamics model (control barrier functions solved as a quadratic program). The HU-MA'AT watchdog is not an avoidance controller: it is a threshold monitor that stops the setpoint motion of the offending aircraft and hands the decision back to the operator or mission coordinator.

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Velocity obstacles [1]	Choose a velocity outside the cone of velocities that would collide with each obstacle within a horizon.	No collision with obstacles that keep their current velocity; feasibility is enforced through the admissible velocities set by acceleration limits.	Ours does not use velocities of peers and does not choose an avoiding velocity; it only freezes the setpoint.
ORCA [2]	Each agent takes half of the responsibility for avoiding each neighbour; one low-dimensional linear program per agent and cycle, no communication.	Collision-free motion for holonomic agents when all agents run ORCA.	Ours is not reciprocal: only an aircraft in FORMATION, MOVE or TAKEOFF reacts; a HOVER, TURN or SPIN peer does nothing.
Control barrier functions, QP safety filter [3]; multi-robot safety barrier certificates [4]	Minimally modify a nominal controller so that a barrier function of the pairwise distance stays non-negative; centralized or decentralized QP each cycle.	Forward invariance of the safe set for the modelled dynamics and input bounds (formal guarantee within the model).	Ours has no dynamics model and no notion of braking distance; it reacts only after the distance is already below 0.82 m.
HU-MA'AT proximity watchdog (this document)	3D distance below 0.82 m at a 10 Hz tick ⇒ freeze setpoint, zero twist, mode HOVER (or GROUNDED below 0.30 m), event.	Only the properties of Section 7; no collision-avoidance guarantee.	—

Where ours is weaker. On every safety property the three established methods establish under their assumptions, ours offers nothing comparable: it is reactive rather than predictive, unilateral rather than reciprocal, and blind between ticks. The live run of 2026-11-12 (Section 8) shows the consequence directly: after

the holds the observed separation still fell to 0.633 m and 0.551 m, well below the 0.82 m hold distance. **What it does provide.** It acts on observed odometry rather than on predicted plans, it never interrupts a landing, its trigger and effect are logged as a single event, and it is small enough to review line by line. The current mitigation is upstream (larger planned clearance), not a better runtime law.

10. Relation to other algorithms

Runtime backstop for Algorithms 5, 6 and 8; tick structure shared with Algorithm 9. Algorithm 11 detects a hold indirectly (live setpoint deviating from the checked plan) and stops the mission. The 0.82 m value is also the alert reference in Algorithm 17; the strategy veto of Algorithm 14 uses the larger planned-clearance margin VETO_M = 1.10 m (Section 8).

11. References

1. P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998, doi:10.1177/027836499801700706. `https://doi.org/10.1177/027836499801700706`
2. J. van den Berg, S. J. Guy, M. Lin and D. Manocha, "Reciprocal n-body collision avoidance," *Robotics Research (Springer Tracts in Advanced Robotics, vol. 70)*, 2011, doi:10.1007/978-3-642-19457-3_1. `https://gamma.cs.unc.edu/ORCA/`
3. A. D. Ames, X. Xu, J. W. Grizzle and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi:10.1109/TAC.2016.2638961. `https://arxiv.org/abs/1609.06408`
4. L. Wang, A. D. Ames and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Transactions on Robotics*, 2017, doi:10.1109/TRO.2017.2659727. `https://resolver.caltech.edu/CaltechAUTHORS:20170215-165434009`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.